

Advanced Error Handling in JavaScript

JavaScript-এর basic error handling জানলে আমরা `try...catch`, `throw`, `finally` ইত্যাদি ব্যবহার করতে পারি।

কিন্তু real-world application-এ error handling আরও structured হওয়া দরকার।

একটি production application-এ আমাদের জানতে হয়:

- কীভাবে নিজের custom error তৈরি করব?
- Error-এর original cause কীভাবে preserve করব?
- Promise rejection globally কীভাবে detect করব?
- Node.js application-এর global errors কীভাবে handle করব?
- একাধিক error একসাথে কীভাবে represent করব?
- কোন error recoverable আর কোন error fatal?
- কোথায় error handle করব আর কোথায় শুধু propagate করব?

এই chapter-এ এগুলোই শিখব।

1. Advanced Error Handling Mental Model

একটি application-এ error সাধারণত কয়েকটি layer-এর মধ্য দিয়ে যায়:

```
Operation
  ↓
Error occurs
  ↓
Error object created
  ↓
Error propagates
  ↓
Local handling
  ↓
Logging / recovery
  ↓
Global fallback (if needed)
```

উদাহরণ:

```
Database
  ↓
DatabaseError
  ↓
```

```
Service layer
  ↓
cause preserved
  ↓
API layer
  ↓
HTTP response
  ↓
Global logger
```

ভালো error handling-এর লক্ষ্য শুধু error "ধরা" না।

লক্ষ্য হলো:

1. Error-এর context preserve করা
2. Error-এর type বোঝা
3. Appropriate layer-এ handle করা
4. Useful information log করা
5. User-কে safe message দেওয়া
6. Application-এর stability বজায় রাখা

2. Custom Error Classes

JavaScript-এর built-in `Error` class সব ধরনের application error-এর জন্য যথেষ্ট expressive নাও হতে পারে।

আমরা নিজেদের error class তৈরি করতে পারি।

Basic Custom Error

```
class ValidationError extends Error {
  constructor(message) {
    super(message);

    this.name = "ValidationError";
  }
}

throw new ValidationError("Email is required");
```

এখানে:

ValidationError



Error



Object

ValidationError হলো Error -এর subclass।

3. Why Custom Error Classes?

Custom error class ব্যবহার করলে আমরা error-এর type আলাদা করতে পারি।

```
class ValidationError extends Error {  
  constructor(message) {  
    super(message);  
    this.name = "ValidationError";  
  }  
}  
  
class NotFoundError extends Error {  
  constructor(message) {  
    super(message);  
    this.name = "NotFoundError";  
  }  
}
```

তারপর:

```
throw new ValidationError("Invalid email");
```

অথবা:

```
throw new NotFoundError("User not found");
```

এখন caller বুঝতে পারবে error-এর ধরন কী।

4. Handling Custom Errors

Custom error class-এর সবচেয়ে বড় সুবিধা হলো `instanceof` দিয়ে type check করা যায়।

```
try {
  throw new ValidationError("Invalid email");
} catch (error) {
  if (error instanceof ValidationError) {
    console.log("Validation problem");
  }
}
```

Multiple error types:

```
try {
  doSomething();
} catch (error) {
  if (error instanceof ValidationError) {
    // Handle validation
  } else if (error instanceof NotFoundError) {
    // Handle not found
  } else {
    // Unknown error
  }
}
```

5. Custom Error with Extra Properties

Custom error-এর মধ্যে additional information রাখা যায়।

```
class HttpError extends Error {
  constructor(message, statusCode) {
    super(message);

    this.name = "HttpError";
    this.statusCode = statusCode;
  }
}
```

ব্যবহার:

```
throw new HttpError("User not found", 404);
```

তারপর:

```
try {  
  throw new HttpError("User not found", 404);  
} catch (error) {  
  console.log(error.message);  
  console.log(error.statusCode);  
}
```

Output:

```
User not found  
404
```

6. Custom Error Design

একটি production-level custom error class এমন হতে পারে:

```
class AppError extends Error {  
  constructor(message, options = {}) {  
    super(message, options);  
  
    this.name = "AppError";  
    this.code = options.code;  
    this.statusCode = options.statusCode;  
  }  
}
```

ব্যবহার:

```
throw new AppError("User not found", {  
  code: "USER_NOT_FOUND",  
  statusCode: 404  
});
```

এখন error-এর মধ্যে থাকবে:

```
{  
  message: "User not found",  
  name: "AppError",  
  code: "USER_NOT_FOUND",  
}
```

```
    statusCode: 404
  }
```

7. Preserve the Stack Trace

Error থেকে inherit করলে:

```
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}
```

JavaScript সাধারণত stack trace তৈরি করে।

```
try {
  throw new ValidationError("Invalid input");
} catch (error) {
  console.log(error.stack);
}
```

Stack trace debugging-এর জন্য অত্যন্ত গুরুত্বপূর্ণ।

8. Error Inheritance

একাধিক specialized error তৈরি করা যায়।

```
class AppError extends Error {
  constructor(message) {
    super(message);
    this.name = "AppError";
  }
}

class ValidationError extends AppError {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}
```

```

    }
  }

  class AuthenticationError extends AppError {
    constructor(message) {
      super(message);
      this.name = "AuthenticationError";
    }
  }
}

```

এখন:

```

const error = new ValidationError("Invalid email");

console.log(error instanceof ValidationError);
console.log(error instanceof AppError);
console.log(error instanceof Error);

```

Output:

```

true
true
true

```

কারণ:

```

ValidationError
  ↓
AppError
  ↓
Error

```

9. Error Chaining

Real-world application-এ একটি error অন্য error-এর কারণে হতে পারে।

ধরো:

```

Database error
  ↓

```

```
User service error
  ↓
API error
```

আমরা original error হারিয়ে ফেলতে চাই না।

JavaScript-এর `Error` constructor-এর `cause` option এজন্য useful।

10. `Error.cause`

```
try {
  databaseOperation();
} catch (error) {
  throw new Error("Failed to load user", {
    cause: error
  });
}
```

এখন নতুন error-এর মধ্যে original error থাকবে:

```
error.cause
```

11. Why `cause` is Important

ধরো original error:

```
const databaseError = new Error("Connection refused");
```

তারপর:

```
throw new Error("Failed to fetch user", {
  cause: databaseError
});
```

এখন:

```

try {
  throw new Error("Failed to fetch user", {
    cause: databaseError
  });
} catch (error) {
  console.log(error.message);
  console.log(error.cause.message);
}

```

Output:

```

Failed to fetch user
Connection refused

```

এতে high-level context এবং low-level root cause দুটোই থাকে।

12. Error Chaining Across Layers

Production application-এ:

```

async function getUser(id) {
  try {
    return await database.findUser(id);
  } catch (error) {
    throw new Error("Failed to get user", {
      cause: error
    });
  }
}

```

তারপর service layer:

```

async function getUserProfile(id) {
  try {
    return await getUser(id);
  } catch (error) {
    throw new Error("Failed to load user profile", {
      cause: error
    });
  }
}

```

```
}  
}
```

এখন chain:

```
Failed to load user profile  
  ↓ cause  
Failed to get user  
  ↓ cause  
Database connection error
```

এটি debugging-এর জন্য অনেক useful।

13. Custom Error + Cause

দুটো একসাথেও ব্যবহার করা যায়।

```
class DatabaseError extends Error {  
  constructor(message, options = {}) {  
    super(message, options);  
  
    this.name = "DatabaseError";  
  }  
}
```

তারপর:

```
try {  
  connectToDatabase();  
} catch (error) {  
  throw new DatabaseError("Database operation failed", {  
    cause: error  
  });  
}
```

14. Error Cause vs Error Message

খারাপ approach:

```
throw new Error(  
  `Database failed because ${error.message}`  
);
```

এতে original error object হারিয়ে যায়।

Better:

```
throw new Error("Database operation failed", {  
  cause: error  
});
```

কারণ:

```
error.cause
```

দিয়ে original error পাওয়া যায়।

15. Unhandled Promise Rejection

Promise reject হয়েছে কিন্তু কেউ সেটা handle করেনি।

```
Promise.reject(new Error("Something went wrong"));
```

এটি একটি unhandled rejection তৈরি করতে পারে।

আরেকটি example:

```
async function test() {  
  throw new Error("Something went wrong");  
}  
  
test();
```

এখানে returned Promise reject হবে।

কিন্তু আমরা যদি সেটি handle না করি:

```
test();
```

তাহলে rejection unhandled হতে পারে।

16. Handled vs Unhandled Rejection

Handled:

```
test().catch(error => {  
  console.error(error);  
});
```

Unhandled:

```
test();
```

অথবা:

```
const promise = Promise.reject(  
  new Error("Failed")  
);
```

কিন্তু:

```
promise.catch(...);
```

দেওয়া হয়নি।

17. Node.js `process.on("unhandledRejection")`

Node.js application-এ unhandled Promise rejection observe করার জন্য:

```
process.on("unhandledRejection", (reason, promise) => {  
  console.error("Unhandled rejection:", reason);  
});
```

এখানে:

```
reason
```

সাধারণত rejection-এর error/value।

আর:

```
promise
```

হলো rejected Promise।

18. Example

```
process.on("unhandledRejection", (reason) => {  
  console.error("Unhandled Promise Rejection");  
  console.error(reason);  
});  
  
Promise.reject(new Error("Database failed"));
```

এটি global-level monitoring-এর জন্য useful।

19. Important: Global Handler Is Not Normal Error Handling

এটা খুব গুরুত্বপূর্ণ।

এভাবে লিখে:

```
process.on("unhandledRejection", () => {});
```

আমরা প্রতিটি Promise-এর `.catch()` বাদ দিতে পারি না।

Global handler হলো:

Safety net

Primary error handling mechanism না।

Better:

```
try {  
  await operation();  
} catch (error) {  
  handleError(error);  
}
```

Global handler:

```
process.on("unhandledRejection", ...)
```

হলো শেষের fallback/monitoring layer।

20. Floating Promises

Floating Promise হলো এমন Promise যার result/rejection properly handle করা হয়নি।

```
async function saveData() {  
  // ...  
}  
  
saveData();
```

যদি `saveData()` reject করতে পারে, তাহলে সেটা floating Promise হতে পারে।

Better:

```
await saveData();
```

অথবা:

```
saveData().catch(handleError);
```

21. Intentional Fire-and-Forget

কখনো আমরা ইচ্ছাকৃতভাবে Promise-এর জন্য wait করতে চাই না।

যেমন:

```
sendAnalytics();
```

কিন্তু যদি এটি fail করতে পারে, তাহলে rejection handle করা উচিত।

```
sendAnalytics().catch(error => {  
  console.error("Analytics failed:", error);  
});
```

অথবা:

```
void sendAnalytics().catch(error => {  
  console.error(error);  
});
```

`void` এখানে developer intent পরীক্ষার করে:

আমি এই Promise-এর result intentionally await করছি না।

22. Global Error Handlers

Global error handling application-এর একেবারে শেষ safety layer।

Node.js-এ প্রধান দুইটি event গুরুত্বপূর্ণ:

```
process.on("unhandledRejection", ...)
```

এবং:

```
process.on("uncaughtException", ...)
```

23. uncaughtException

যখন synchronous exception কোথাও throw হয় কিন্তু কোনো `try...catch` সেটি catch করে না:

```
throw new Error("Fatal error");
```

Node.js process-level handler দিয়ে observe করা যায়:

```
process.on("uncaughtException", (error) => {  
  console.error("Uncaught exception:", error);  
});
```

24. uncaughtException Is Dangerous

এখানে একটি খুব important production concept আছে।

`uncaughtException` handler পাওয়া মানেই application safe হয়ে গেছে—এমন না।

কারণ uncaught exception application-এর internal state inconsistent করে দিতে পারে।

তাই:

```
process.on("uncaughtException", error => {  
  console.error(error);  
});
```

দিয়ে application-কে indefinitely চালিয়ে যাওয়া সবসময় safe না।

অনেক production architecture-এ:

```
uncaughtException  
  ↓  
log  
  ↓  
cleanup  
  ↓  
graceful shutdown
```

↓
process manager restarts app

এই pattern বেশি নিরাপদ হতে পারে।

25. Graceful Shutdown

যদি process বন্ধ করতে হয়, আগে প্রয়োজনীয় cleanup করা উচিত।

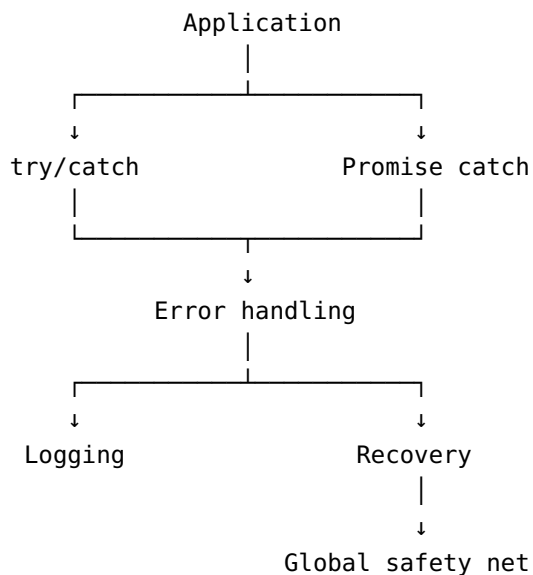
যেমন:

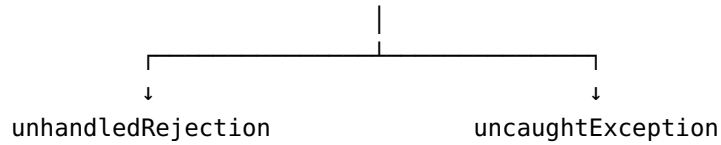
```
process.on("uncaughtException", error => {  
  console.error("Fatal error:", error);  
  
  server.close(() => {  
    process.exit(1);  
  });  
});
```

তবে বাস্তবে cleanup architecture application অনুযায়ী design করতে হবে।

26. Global Error Handling Architecture

একটি Node.js backend-এর conceptual architecture:





27. Browser Global Error Handling

Browser environment-এ Node.js-এর `process` নেই।

Browser-এ:

```
window.addEventListener("error", event => {
  console.error("Global error:", event.error);
});
```

Promise rejection monitor করতে:

```
window.addEventListener(
  "unhandledrejection",
  event => {
    console.error(
      "Unhandled rejection:",
      event.reason
    );
  }
);
```

এগুলো global monitoring-এর জন্য useful।

28. Local Error Handling vs Global Error Handling

Local Handling	Global Handling
Specific operation	Application-wide fallback
Recovery possible	Monitoring/fallback
Business logic aware	Generic
Usually preferred	Last safety net

Local Handling

Global Handling

Specific error context

Broad error context

Rule:

Handle an error at the lowest layer that has enough information to recover from it.

29. Don't Catch Errors Just to Rethrow Them

Bad:

```
try {
  doSomething();
} catch (error) {
  throw error;
}
```

এখানে কোনো value add হচ্ছে না।

যদি context add করতে হয়:

```
try {
  doSomething();
} catch (error) {
  throw new Error("Failed to process order", {
    cause: error
  });
}
```

এটা meaningful।

30. AggregateError

কখনো একটি operation-এর মধ্যে multiple errors হতে পারে।

JavaScript এজন্য `AggregateError` দেয়।

```
const error = new AggregateError(
  [
```

```
    new Error("Database failed"),
    new Error("Cache failed"),
    new Error("API failed")
  ],
  "Multiple operations failed"
);
```

এখানে:

```
error.errors
```

এর মধ্যে individual errors পাওয়া যাবে।

31. AggregateError Example

```
const errors = [
  new Error("Task 1 failed"),
  new Error("Task 2 failed"),
  new Error("Task 3 failed")
];

throw new AggregateError(
  errors,
  "Multiple tasks failed"
);
```

Handle:

```
try {
  throw new AggregateError(
    [
      new Error("Task 1 failed"),
      new Error("Task 2 failed")
    ],
    "Multiple tasks failed"
  );
} catch (error) {
  if (error instanceof AggregateError) {
    for (const individualError of error.errors) {
      console.error(individualError.message);
    }
  }
}
```

```
}  
}
```

32. Promise.any() and AggregateError

Promise.any() একটি important example।

```
Promise.any([  
  Promise.reject(new Error("Server A failed")),  
  Promise.reject(new Error("Server B failed")),  
  Promise.reject(new Error("Server C failed"))  
)  
.catch(error => {  
  console.log(error instanceof AggregateError);  
});
```

সব Promise reject করলে Promise.any() একটি AggregateError দিয়ে reject করে।

33. AggregateError vs Normal Error

Normal:

```
new Error("One operation failed");
```

Aggregate:

```
new AggregateError(  
  [  
    error1,  
    error2,  
    error3  
  ],  
  "Multiple operations failed"  
);
```

Mental model:

```
Error
└─ One error

AggregateError
├─ Error 1
├─ Error 2
└─ Error 3
```

34. Fail-Fast vs Collect-All Errors

দুই ধরনের architecture দেখা যায়।

Fail-Fast

প্রথম critical error পেলেই stop।

```
await Promise.all([
  task1(),
  task2(),
  task3()
]);
```

একটি reject করলে `Promise.all()` reject করবে।

Collect-All

সব operation complete করে errors collect করা।

```
const results = await Promise.allSettled([
  task1(),
  task2(),
  task3()
]);
```

তারপর errors collect:

```
const errors = results
  .filter(result => result.status === "rejected")
  .map(result => result.reason);
```

তারপর চাইলে:

```
if (errors.length > 0) {
  throw new AggregateError(
    errors,
    "Some tasks failed"
  );
}
```

35. Practical `AggregateError` Pattern

```
async function runTasks(tasks) {
  const results = await Promise.allSettled(
    tasks.map(task => task())
  );

  const errors = results
    .filter(result => result.status === "rejected")
    .map(result => result.reason);

  if (errors.length > 0) {
    throw new AggregateError(
      errors,
      `${errors.length} task(s) failed`
    );
  }

  return results.map(result => result.value);
}
```

এটি useful যখন:

"সব task চালাও, তারপর failed task-গুলোর information একসাথে দাও।"

36. Error Classification

Production application-এ error-কে broadly classify করা যায়।

Validation Error

User input ভুল।

Invalid email

Authentication Error

User authenticate করতে পারেনি।

Invalid credentials

Authorization Error

User-এর permission নেই।

Access denied

Not Found Error

Resource পাওয়া যায়নি।

User not found

Network Error

Network communication problem।

Database Error

Database operation failure।

Configuration Error

Application configuration problem।

Programming Error

Code-এর bug।

37. Recoverable vs Non-Recoverable Errors

সব error একইভাবে handle করা উচিত না।

Recoverable

```
Temporary network failure
Rate limit
Timeout
```

Retry করা যেতে পারে।

Non-Recoverable

```
Invalid application state
Broken invariant
Programming bug
Corrupted internal state
```

এগুলোর জন্য application shutdown/restart প্রয়োজন হতে পারে।

38. Operational vs Programming Errors

এটি backend development-এ খুব useful distinction।

Operational Error

Application-এর expected environment problem।

উদাহরণ:

```
Database temporarily unavailable
Network timeout
```

```
Invalid user input  
File not found
```

Application এগুলো handle করতে পারে।

Programming Error

Code-এর bug।

উদাহরণ:

```
undefinedUser.name
```

অথবা:

```
someVariableThatDoesNotExist();
```

এগুলো সাধারণত code fix করার signal।

39. Error Boundary Concept

Error handling-এর একটি useful architecture:

```
Low-level layer  
  ↓  
Create detailed error  
  ↓  
Service layer  
  ↓  
Add context with cause  
  ↓  
Controller/API layer  
  ↓  
Convert to safe response  
  ↓  
Global logger
```

একটি layer-এর কাজ:

নিজের জন্য context add করা।

40. Never Leak Internal Errors to Users

Bad:

```
res.status(500).json({  
  message: error.stack  
});
```

এতে internal information leak হতে পারে।

Better:

```
res.status(500).json({  
  message: "Internal server error"  
});
```

আর server-side log:

```
console.error(error);
```

অর্থাৎ:

```
User → Safe message  
Server → Detailed error
```

41. Logging Error Chains

যদি:

```
try {  
  databaseOperation();  
} catch (error) {  
  throw new Error("User loading failed", {  
    cause: error  
  });  
}
```

তাহলে logging system ideally chain-টাও preserve করবে।

Conceptually:

```
User loading failed
↓
Database operation failed
↓
Connection refused
```

এতে root cause খুঁজে বের করা সহজ হয়।

42. Common Mistake: Swallowing Errors

থারাপ:

```
try {
  doSomething();
} catch (error) {
  console.log(error);
}
```

তারপর function silently return করে।

এতে caller বুঝতে পারে না operation fail করেছে।

Better:

```
try {
  doSomething();
} catch (error) {
  console.error(error);

  throw error;
}
```

অথবা meaningful transformation:

```
throw new AppError("Operation failed", {  
  cause: error  
});
```

43. Common Mistake: Generic Error Everywhere

সব জায়গায়:

```
throw new Error("Something went wrong");
```

করলে debugging কঠিন হয়।

Better:

```
throw new ValidationError("Email is invalid");
```

```
throw new NotFoundError("User not found");
```

```
throw new AuthenticationError("Invalid credentials");
```

44. Common Mistake: Using Global Handler as Primary Handler

এটা ভুল:

```
process.on("unhandledRejection", error => {  
  console.log(error);  
});
```

এবং ধরে নেওয়া:

"সব Promise এখানেই handle হবে।"

না।

Primary:

```
try {
  await operation();
} catch (error) {
  // Handle appropriately
}
```

Global handler:

Last safety net

45. Common Mistake: Ignoring AggregateError.errors

এটা করলে:

```
catch (error) {
  console.log(error.message);
}
```

তুমি শুধু:

Multiple operations failed

পাবে।

কিন্তু individual errors দেখতে:

```
catch (error) {
  if (error instanceof AggregateError) {
    console.log(error.errors);
  }
}
```

46. Custom Error Factory Pattern

কখনো centralized factory ব্যবহার করা যায়:

```
function createHttpError(  
  statusCode,  
  message,  
  options = {}  
) {  
  const error = new Error(message, options);  
  
  error.statusCode = statusCode;  
  
  return error;  
}
```

ব্যবহার:

```
throw createHttpError(  
  404,  
  "User not found"  
);
```

তবে বড় application-এ custom error classes অনেক সময় বেশি expressive ।

47. Custom Error Classes vs Error Codes

দুই approach দেখা যায় ।

Class-based

```
if (error instanceof ValidationError) {  
  // ...  
}
```

Code-based

```
if (error.code === "USER_NOT_FOUND") {  
  // ...  
}
```

অনেক production system-এ দুটো একসাথেও থাকে:

```
class NotFoundError extends AppError {  
  constructor(message) {  
    super(message);  
  
    this.name = "NotFoundError";  
    this.code = "NOT_FOUND";  
    this.statusCode = 404;  
  }  
}
```

48. A Production-Style Error Base Class

```
class AppError extends Error {  
  constructor(message, {  
    code = "INTERNAL_ERROR",  
    statusCode = 500,  
    cause  
  } = {}) {  
    super(message, { cause });  
  
    this.name = "AppError";  
    this.code = code;  
    this.statusCode = statusCode;  
  }  
}
```

Specialized error:

```
class NotFoundError extends AppError {  
  constructor(message = "Resource not found") {  
    super(message, {  
      code: "NOT_FOUND",  
      statusCode: 404  
    })  
  }  
}
```

```
});  
  
    this.name = "NotFoundError";  
  }  
}
```

ব্যবহার:

```
throw new NotFoundError("User not found");
```

49. Error Handling in Async Functions

```
async function getUser() {  
  try {  
    const response = await fetch("/api/user");  
  
    if (!response.ok) {  
      throw new Error("Request failed");  
    }  
  
    return await response.json();  
  } catch (error) {  
    throw new Error("Failed to load user", {  
      cause: error  
    });  
  }  
}
```

Caller:

```
try {  
  const user = await getUser();  
} catch (error) {  
  console.error(error);  
}
```

এখানে error responsibility layer-by-layer ভাগ হয়েছে।

50. Error Handling Principle

সবচেয়ে useful rule:

Don't catch an error unless you can handle it, enrich it, transform it, log it appropriately, or recover from it.

শুধু:

```
catch (error) {  
  console.log(error);  
}
```

করার দরকার নেই।

অনেক সময় error propagate করাই better।

51. Complete Error Flow Example

```
class DatabaseError extends Error {  
  constructor(message, options = {}) {  
    super(message, options);  
  
    this.name = "DatabaseError";  
  }  
}  
  
class UserServiceError extends Error {  
  constructor(message, options = {}) {  
    super(message, options);  
  
    this.name = "UserServiceError";  
  }  
}
```

Database layer:

```
async function findUser(id) {  
  try {  
    return await database.find(id);  
  } catch (error) {
```

```

        throw new DatabaseError(
            "Database query failed",
            {
                cause: error
            }
        );
    }
}

```

Service layer:

```

async function getUser(id) {
    try {
        return await findUser(id);
    } catch (error) {
        throw new UserServiceError(
            "Failed to load user",
            {
                cause: error
            }
        );
    }
}

```

Controller:

```

try {
    const user = await getUser(id);

    return user;
} catch (error) {
    console.error(error);

    throw error;
}

```

Concept:

```

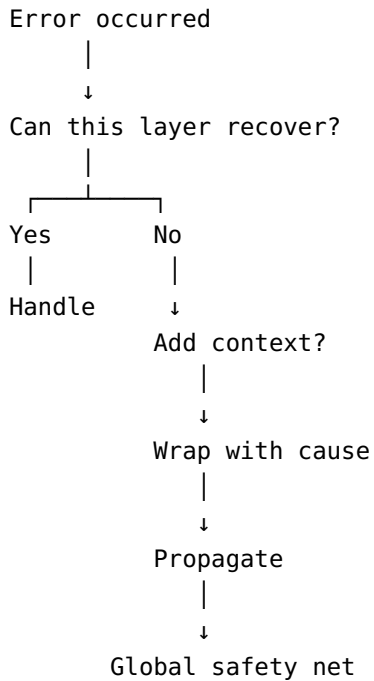
Original DB error
    ↓
DatabaseError
    ↓ cause
UserServiceError
    ↓

```

Controller
↓
Logger / Response

এটাই error chaining-এর practical power ।

52. Advanced Error Handling Decision Tree



53. Important Rules to Remember

Rule 1

Custom errors তৈরি করো যখন error type আলাদা করা দরকার ।

Rule 2

Original error হারিয়ে ফেলো না ।

Use:

```
{ cause: error }
```

Rule 3

Promise rejection অবশ্যই intentionally handle করে।

```
await promise;
```

অথবা:

```
promise.catch(...)
```

Rule 4

Global handlers হলো safety net।

Primary error handling নয়।

Rule 5

`uncaughtException` -এর পরে application-এর state নিয়ে সতর্ক থাকো।

Rule 6

Multiple errors represent করতে:

```
AggregateError
```

ব্যবহার করে।

Rule 7

User-কে internal stack trace দেখিও না।

Rule 8

Error swallow করে না।

Rule 9

যে layer recover করতে পারে না, সে error propagate করতে পারে।

Rule 10

Context যোগ করলে `cause` ব্যবহার করো।

54. Practice — Easy

Task 1

একটি custom `ValidationError` class তৈরি করো।

```
throw new ValidationError("Email is required");
```

Task 2

`NotFoundError` তৈরি করো এবং `instanceof` দিয়ে check করো।

Task 3

একটি error-এর cause তৈরি করো:

```
Database failed
  ↓
Failed to load user
```

Task 4

একটি rejected Promise-এর `.catch()` ব্যবহার করো।

55. Practice — Medium

Task 5

এই error hierarchy তৈরি করো:

```
Error
↓
AppError
├─ ValidationError
├─ NotFoundError
└─ AuthenticationError
```

Task 6

`statusCode` এবং `code` support করে এমন `AppError` তৈরি করো।

Example:

```
new NotFoundError("User not found");
```

যাতে:

```
error.statusCode === 404
```

হয়।

Task 7

`Promise.allSettled()` ব্যবহার করে সব failed operations collect করো এবং শেষে `AggregateError` throw করো।

56. Practice — Advanced

Task 8

এই architecture implement করো:

```
Database
↓
DatabaseError
↓ cause
ServiceError
```

↓ cause
Controller

Task 9

একটি global Node.js handler তৈরি করে:

```
process.on("unhandledRejection", ...)
```

এবং intentionally unhandled rejection তৈরি করে observe করে।

Task 10

একটি graceful shutdown strategy design করে:

```
Fatal error  
↓  
Log  
↓  
Stop accepting new requests  
↓  
Cleanup  
↓  
Exit
```

57. Output Prediction

Question 1

```
class MyError extends Error {}  
  
try {  
  throw new MyError("Oops");  
} catch (error) {  
  console.log(error instanceof MyError);  
  console.log(error instanceof Error);  
}
```

Output কী?

Question 2

```
const original = new Error("Database failed");

const wrapped = new Error("User loading failed", {
  cause: original
});

console.log(wrapped.message);
console.log(wrapped.cause.message);
```

Output কী?

Question 3

```
Promise.any([
  Promise.reject("A"),
  Promise.reject("B")
]).catch(error => {
  console.log(error instanceof AggregateError);
  console.log(error.errors);
});
```

Output কী?

58. Interview Questions

Beginner

1. Custom Error class কেন ব্যবহার করা হয়?
2. `Error` extend করার উদ্দেশ্য কী?
3. `instanceof` দিয়ে error type কীভাবে check করবে?
4. Error chaining কী?
5. `error.cause` কী?

Intermediate

1. `unhandledRejection` কী?
2. Floating Promise কী?

3. Global error handler কেন দরকার?
4. `uncaughtException` এবং `unhandledRejection` -এর পার্থক্য কী?
5. Global handler কি local error handling replace করতে পারে?

Advanced

1. `AggregateError` কী?
2. `Promise.any()` কখন `AggregateError` throw করে?
3. Fail-fast vs collect-all কী?
4. Error cause preserve করা কেন গুরুত্বপূর্ণ?
5. Operational error vs programming error কী?
6. কেন `uncaughtException` -এর পরে process restart করা লাগতে পারে?
7. Error swallow করা কেন dangerous?
8. Custom Error class এবং error code—কখন কোনটা ব্যবহার করবে?
9. User-facing error এবং internal error আলাদা রাখা কেন গুরুত্বপূর্ণ?
10. Error handling architecture কীভাবে design করবে?

59. Quick Revision

Custom Error

↓

```
class MyError extends Error
```

Error chaining

↓

```
new Error(message, { cause: error })
```

Original error

↓

```
error.cause
```

Unhandled Promise

↓

```
process.on("unhandledRejection", ...)
```

Uncaught synchronous exception

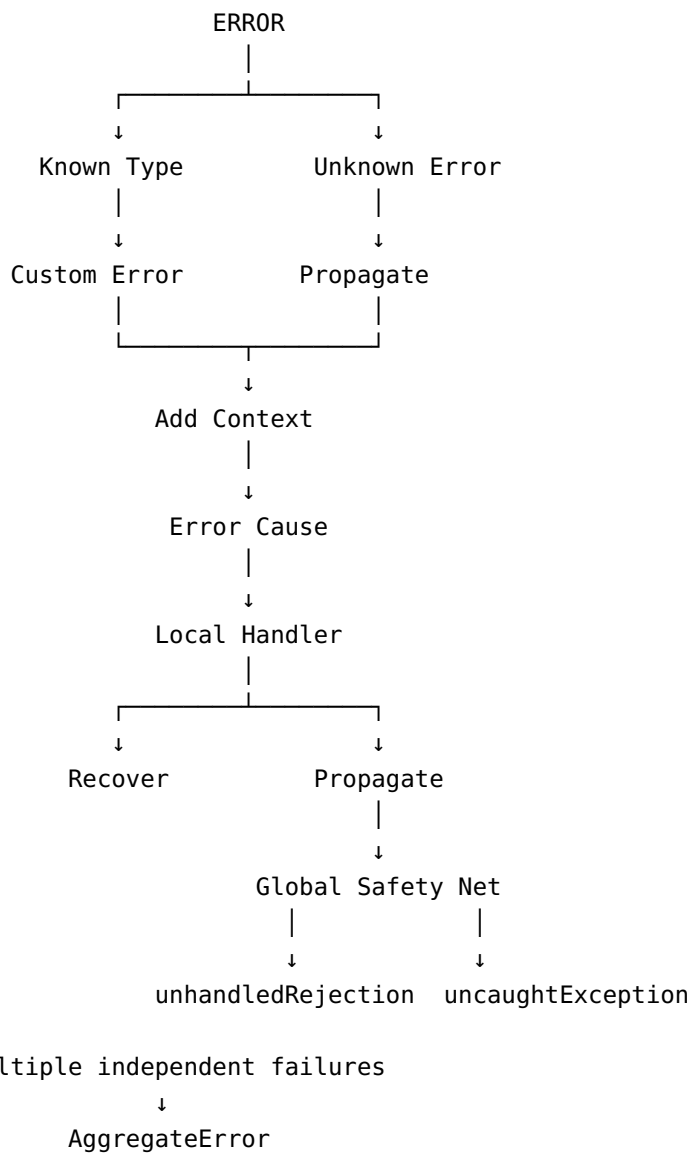
↓

```
process.on("uncaughtException", ...)
```

Multiple errors
↓
AggregateError
↓
error.errors

60. Final Mental Model

Advanced Error Handling-কে এভাবে মনে রাখো:



সবচেয়ে important পাঁচটি জিনিস:

1. Custom Error
2. Error cause
3. Promise rejection handling
4. Global error safety net
5. AggregateError

আর production-level golden rule:

Catch errors where you can meaningfully handle or enrich them; otherwise let them propagate.